

Audit de Base de Données

EasyQual — Plateforme de gestion Qualiopi

Projet Supabase	gxworwhpcyfuqwuxocxx
Moteur	PostgreSQL 17.6 — eu-west-1
Environnement	main · PRODUCTION
Schéma	public — 22 tables
Date de l'audit	30 juin 2026
Périmètre	Schéma, RLS, RPC, GRANT, index

1. Synthèse exécutive

Le **modèle relationnel est solide** (clés étrangères cohérentes, cascades, RPC transactionnelle avec verrou `FOR UPDATE`). En revanche la **couche de sécurité présente 5 failles critiques** exploitables, dont une permettant la **fraude financière par un utilisateur non authentifié** et une permettant l'**auto-promotion en administrateur**. Une remédiation urgente est requise avant toute exploitation en production.

Sévérité	Nombre	Domaine principal	Délai conseillé
CRITIQUE	5	Privilèges, secrets, mots de passe	< 24 h
MAJEUR	5	RLS / contrôle d'accès	1–2 semaines
MINEUR	5	Intégrité / schéma / perf	Backlog
POSITIF	4	Modèle, RPC, RLS récents	—

Méthodologie

Analyse statique de `supabase/schema.sql` , des 60+ fichiers de `supabase/migrations/` , des fonctions `SECURITY DEFINER` , des instructions `GRANT` / `REVOKE` , des polices RLS et des index, croisée avec le Schema Visualizer de Supabase (état live). Aucune connexion Postgres directe (clés API uniquement) : une vérification live des données (orphelins, doublons SIRET) reste recommandée en complément.

2. Vue d'ensemble du modèle

22 tables dans le schéma `public` . Architecture multi-tenant : un **consultant** crée des **tenants** (organismes de formation), chaque tenant ayant un `owner_id` (compte client `of`). L'isolation des données repose **entièrement sur le RLS**.

Domaine	Tables
Identité & rôles	<code>profiles</code> (lié à <code>auth.users</code>), enum <code>user_role</code> = admin / consultant / of
Facturation / crédits	<code>credits_wallet</code> , <code>transactions</code>
Clients & dossiers	<code>tenants</code> , <code>cases</code> , <code>sessions</code>
Référentiel Qualiopi	<code>criteria</code> (7), <code>indicators</code> (32), <code>qualiopi_audits</code>
Preuves & suivi	<code>evidences</code> , <code>reviews</code> , <code>case_indicator_states</code> , <code>criterion_quiz_uploads</code>
Collaboration	<code>case_messages</code> , <code>case_events</code> , <code>case_notifications</code> , <code>admin_notifications</code>
Divers	<code>reclamations</code> , <code>questionnaires_results</code> , <code>consultant_resources</code> , <code>logs</code> , <code>system_settings</code>

3. Failles critiques

C1 Fraude aux crédits : `add_credits` exécutable par `anon`

migrations/20260226_fix_credits_and_grants.sql

Fonction `SECURITY DEFINER` (bypass RLS, exécutée en `postgres`), **sans aucun contrôle d'identité**, accordée à `anon`. La clé `anon` étant publiée dans le bundle JS, n'importe qui peut appeler :

```
// depuis le navigateur, sans être connecté
supabase.rpc('add_credits', { p_consultant_id: '<uuid>', p_amount: 999999 })
```

Impact : recharge gratuite et illimitée du portefeuille de n'importe quel consultant → contournement total du paiement.

Correctif : `REVOKE EXECUTE ... FROM anon;` · déplacer la recharge dans une Edge Function authentifiée déclenchée par le webhook de paiement · ajouter un contrôle de rôle interne.

C2 `create_case_and_debit` exécutable par `anon`

migrations/20260302_add_client_credentials.sql (+ `GRANT ... TO anon`)

Même schéma de faille : un appelant non authentifié peut déclencher la création de dossiers et le débit du portefeuille de n'importe quel `consultant_id`.

Correctif : `REVOKE ... FROM anon;` et vérifier `auth.uid() = p_consultant_id` à l'intérieur de la fonction.

C3 Auto-promotion en admin via `user_metadata`

schema.sql + nombreuses politiques « Admins view all »

Toutes les politiques admin reposent sur :

```
using ( (auth.jwt() -> 'user_metadata' ->> 'role') = 'admin' )
```

Or `user_metadata` est **modifiable par l'utilisateur lui-même** via `supabase.auth.updateUser({ data:{ role:'admin' } })`. Un client `of` peut donc se promouvoir admin et lire toutes les données de tous les clients (tenants, dossiers, portefeuilles, transactions, profils).

Impact : élévation de privilèges + fuite de confidentialité à l'échelle de toute la plateforme.

Correctif : ne jamais utiliser `user_metadata` pour l'autorisation. Vérifier le rôle via la table `profiles` : `exists (select 1 from profiles where id = auth.uid() and role='admin')` — pattern déjà utilisé correctement dans `system_settings`.

C4 Mots de passe stockés en clair

tenants.initial_password · profiles.temp_password

Des mots de passe en clair sont persistés en base et relus par les scripts (`fix-client-password.mjs` , `fix-password.js`). Combiné à C3, une lecture non autorisée expose des identifiants réutilisables.

Impact : violation RGPD et risque de compromission de comptes clients.

Correctif : supprimer ces colonnes · s'appuyer uniquement sur `auth.users.encrypted_password` (bcrypt) · mot de passe provisoire transmis hors-bande, jamais persisté en clair.

C5 Clé `service_role` en clair dans le dépôt Git

`fix-password.js` (et autres scripts versionnés)

La clé `service_role` (bypass total du RLS) est codée en dur dans des fichiers suivis par Git. Elle est compromise dès que le dépôt est partagé ou poussé.

Correctif immédiat : roter la clé dans le dashboard Supabase · déplacer les secrets en variables d'environnement non versionnées · purger l'historique (`git filter-repo`) · vérifier que `.gitignore` couvre `.env*` .

4. Problèmes majeurs — RLS & accès

#	Problème	Détail & correctif
M1	Policy admin cassée	<code>schema.sql:260-262</code> : <code>using(...)</code> orphelin sans <code>create policy ... for select</code> → erreur SQL si rejoué. <code>schema.sql</code> n'est plus fiable.
M2	RLS chroniquement instable	~15 migrations <code>*_fix_rls_*</code> (récursion, <code>permission denied for auth.users</code> , isolation client). Centraliser via des fonctions helper <code>STABLE</code> .
M3	<code>GRANT ALL ... TO authenticated</code>	Sur <code>cases</code> , <code>tenants</code> , <code>case_messages</code> , <code>case_indicator_states</code> , <code>criterion_quiz_uploads</code> . Tout repose sur la justesse du RLS. Restreindre aux verbes nécessaires.
M4	<code>get_auth_user_id(email)</code> ouvert	<code>SECURITY DEFINER</code> lisant <code>auth.users</code> , accessible aux authentifiés → énumération de comptes. Restreindre l'exécution.
M5	Couverture write incomplète	Politiques <code>INSERT/UPDATE/DELETE</code> hétérogènes selon les tables. Cartographier pour garantir l'absence d'écriture cross-tenant.

5. Problèmes mineurs — intégrité, schéma & performance

#	Problème	Détail
m1	Enum <code>transaction_type</code> incomplet	Défini (<code>'purchase'</code> , <code>'adjustment'</code> , <code>'usage'</code>) mais <code>add_credits</code> insère <code>'recharge'</code> → échec ou incohérence. À aligner.
m2	Indicateurs Qualiopi factices	<code>schema.sql</code> insère <code>'Indicateur 1'...'Indicateur 32'</code> (placeholder, mapping « rough »). Injecter les 32 indicateurs officiels.
m3	Index FK manquants	<code>evidences.case_id</code> , <code>reviews.case_id</code> , <code>transactions.wallet_id</code> , <code>sessions.tenant_id</code> , <code>logs.user_id</code> ... Le RLS filtre par ces colonnes à chaque ligne → scans séquentiels coûteux.
m4	Churn de type sur <code>audit_type</code>	<code>text</code> → <code>text[]</code> plusieurs fois le même jour. Vérifier l'état réellement appliqué en prod.
m5	Doublons SIRET historiques	Deux migrations <code>fix_duplicate_siret</code> → des doublons ont existé malgré <code>unique</code> . Vérifier que la contrainte est active.

6. Points positifs

- + **RPC transactionnelle robuste** : `create_case_and_debit` verrouille le portefeuille (`FOR UPDATE`), vérifie le solde et journalise — atomicité correcte.
- + **Bon pattern RLS déjà présent** dans `system_settings` et `qualiopi_audits` (`exists(... profiles ... role='admin')` et `auth.uid()=consultant_id`) — à généraliser.
- + **Index de performance récents** (`20260626_add_performance_indexes.sql`) couvrant les FK les plus chaudes.
- + **Modèle relationnel cohérent** : `on delete cascade` / `set null` bien posés, contraintes `check` sur les soldes et types d'audit.

7. Plan d'action priorisé

Priorité	Action	Réf.
Immédiat	Roten la clé <code>service_role</code> + purger l'historique Git	C5
Immédiat	<code>REVOKE EXECUTE</code> de <code>add_credits</code> et <code>create_case_and_debit</code> pour <code>anon</code>	C1, C2
Immédiat	Remplacer toutes les politiques <code>user_metadata</code> par un check via <code>profiles</code>	C3
Court terme	Supprimer les colonnes mot de passe en clair + refonte du provisioning	C4
Court terme	Contrôle d'autorisation interne dans les fonctions <code>SECURITY DEFINER</code>	C1, M4

Court terme	Compléter les polices write et restreindre les GRANT ALL	M3, M5
Moyen terme	Régénérer schema.sql , centraliser le RLS via helpers	M1, M2
Moyen terme	Ajouter les index FK manquants ; corriger l'enum ; vrais indicateurs Qualiopi	m1–m3

Fin du rapport — Audit statique réalisé le 30/06/2026. Une vérification live (état réel des polices, index, données orphelines, doublons SIRET) est recommandée via supabase db dump ou un accès Postgres direct.